

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose Files](#) WA_Fn-Us...er-Churn.csv

WA_Fn-UseC_-Telco-Customer-Churn.csv(text/csv) - 977501 bytes, last modified: 6/17/2026 - 100% done
Saving WA_Fn-UseC_-Telco-Customer-Churn.csv to WA_Fn-UseC_-Telco-Customer-Churn.csv

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
%matplotlib inline
```

```
sns.set_style("whitegrid")
```

```
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
```

```
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLine
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service
1	5575-GNVDE	Male	0	No	No	34	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No
3	7795-CFOCW	Male	0	No	No	45	No	No phone service
4	9237-HQITU	Female	0	No	No	2	Yes	No

5 rows × 21 columns

```
df.shape
```

```
(7043, 21)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerID            7043 non-null  object
1   gender                 7043 non-null  object
2   SeniorCitizen          7043 non-null  int64
3   Partner                7043 non-null  object
4   Dependents             7043 non-null  object
5   tenure                 7043 non-null  int64
6   PhoneService           7043 non-null  object
7   MultipleLines          7043 non-null  object
8   InternetService        7043 non-null  object
```

```
9  OnlineSecurity    7043 non-null object
10 OnlineBackup      7043 non-null object
11 DeviceProtection  7043 non-null object
12 TechSupport       7043 non-null object
13 StreamingTV       7043 non-null object
14 StreamingMovies   7043 non-null object
15 Contract          7043 non-null object
16 PaperlessBilling  7043 non-null object
17 PaymentMethod     7043 non-null object
18 MonthlyCharges    7043 non-null float64
19 TotalCharges      7043 non-null object
20 Churn             7043 non-null object
```

dtypes: float64(1), int64(2), object(18)

memory usage: 1.1+ MB

```
df.isnull().sum()
```

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
(df['TotalCharges'] == ' ').sum()
```

```
np.int64(11)
```

```
df['TotalCharges'] = pd.to_numeric(
    df['TotalCharges'],
    errors='coerce'
)
```

```
df['TotalCharges'].dtype
```

```
dtype('float64')
```

```
df['TotalCharges'].isnull().sum()
```

```
np.int64(11)
```

```
df['TotalCharges'].fillna(
    df['TotalCharges'].median(),
    inplace=True
)
```

/tmp/ipykernel_6957/1685388760.py:1: FutureWarning: A value is trying to be set on a copy of
The behavior will change in pandas 3.0. This inplace method will never work because the inter
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: val

```
df['TotalCharges'].fillna(
```

```
df.drop('customerID', axis=1, inplace=True)
```

```
df.head()
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetS
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	Fib

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.isnull().sum()
```

	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(22)
```

```
(df['TotalCharges'] == ' ').sum()
```

```
np.int64(0)
```

```
df['TotalCharges'].isnull().sum()
```

```
np.int64(0)
```

```
(df['TotalCharges'] == ' ').sum()
```

```
np.int64(0)
```

```
df['TotalCharges'] = pd.to_numeric(
    df['TotalCharges'],
    errors='coerce'
)
```

```
df['TotalCharges'].fillna(
    df['TotalCharges'].median(),

```

```
inplace=True  
)
```

/tmp/ipykernel_6957/1321978346.py:6: FutureWarning: A value is trying to be set on a copy of
The behavior will change in pandas 3.0. This inplace method will never work because the inter

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: val

```
df['TotalCharges'].fillna(
```

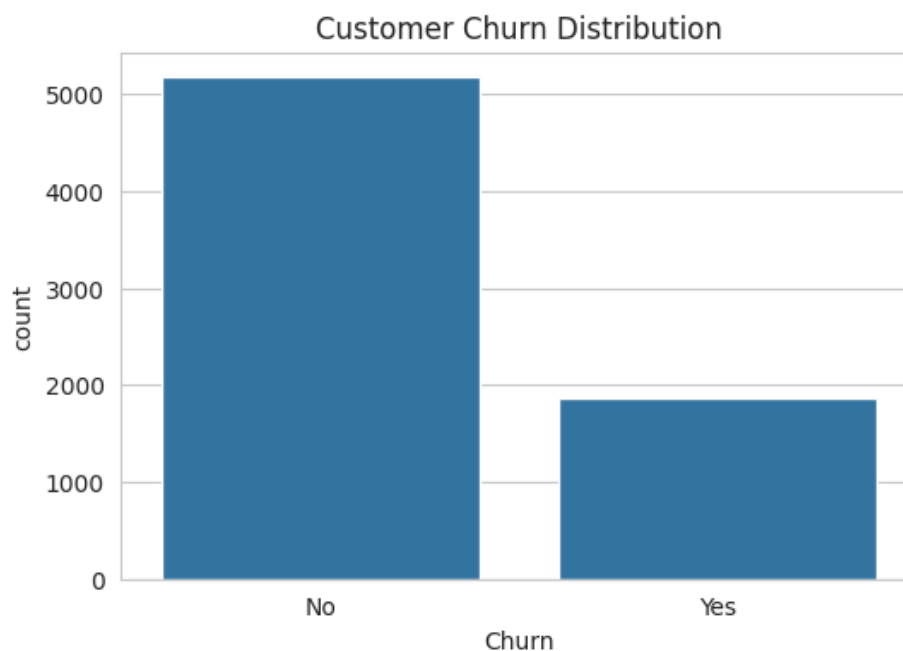
✓ Data Cleaning Summary

The dataset initially contained no missing values in most columns. However, the TotalCharges column was stored as an object datatype due to 22 blank entries. These blank values were converted to NaN and replaced using the median value of the column. The customerID column was removed because it serves only as an identifier and does not contribute to churn analysis. Duplicate records were checked and handled appropriately.

✓ EDA

1.Churn Count Plot

```
plt.figure(figsize=(6,4))  
  
sns.countplot(x='Churn', data=df)  
  
plt.title('Customer Churn Distribution')  
plt.show()
```

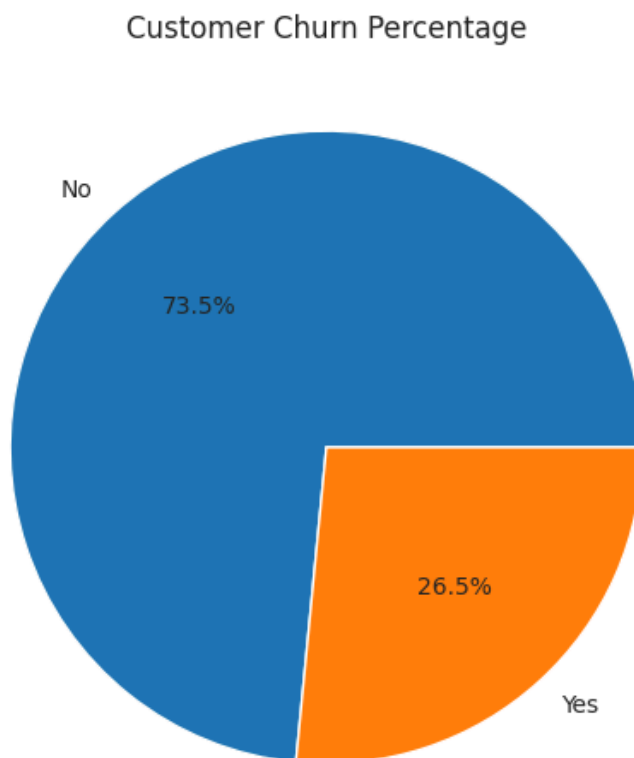


The number of customers who did not churn is significantly higher than those who churned. This indicates that most customers remained with the company, while a smaller proportion discontinued the service.

2.Churn Percentage Pie Chart

```
df['Churn'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%',
    figsize=(6,6)
)

plt.title('Customer Churn Percentage')
plt.ylabel('')
plt.show()
```



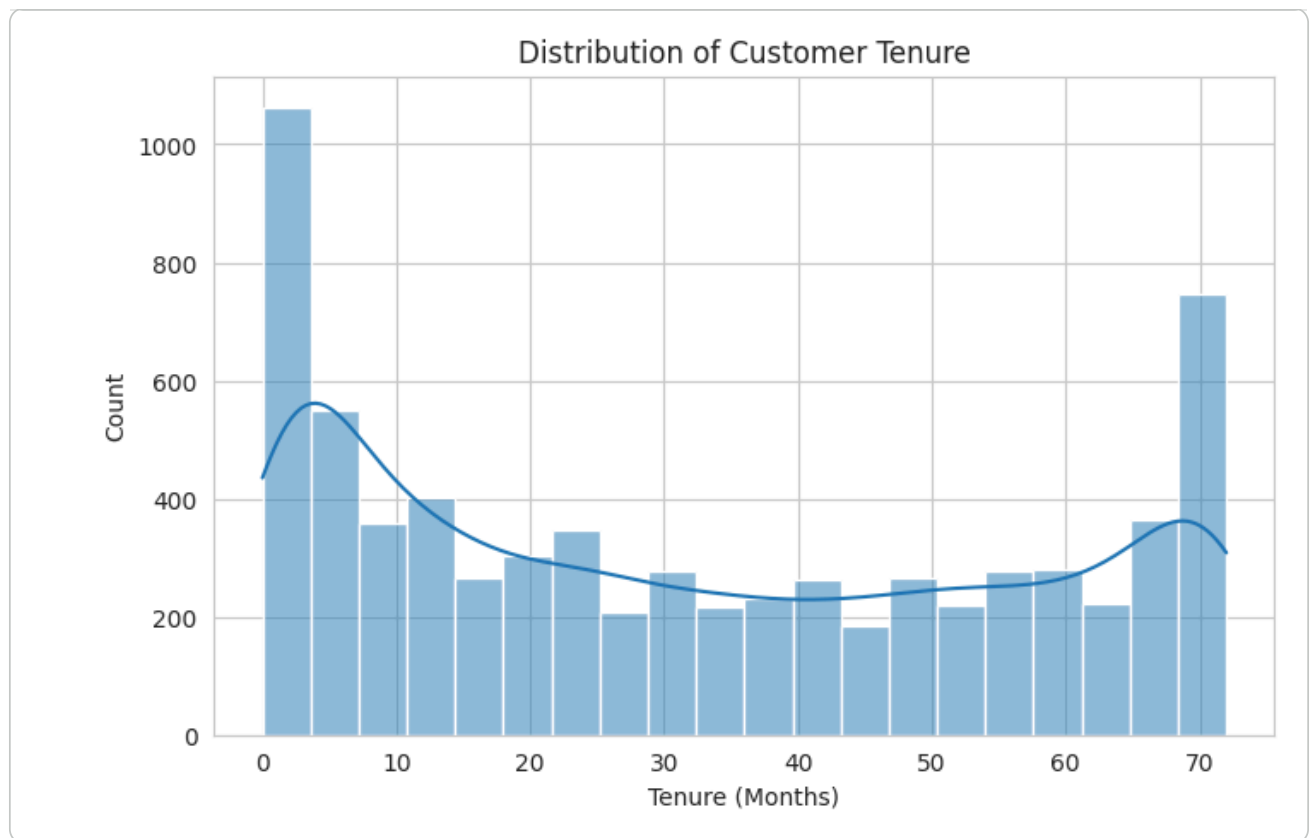
Approximately one-fourth of customers churned, while the majority continued using the company's services.

3.Tenure Histogram

```
plt.figure(figsize=(8,5))

sns.histplot(df['tenure'], bins=20, kde=True)

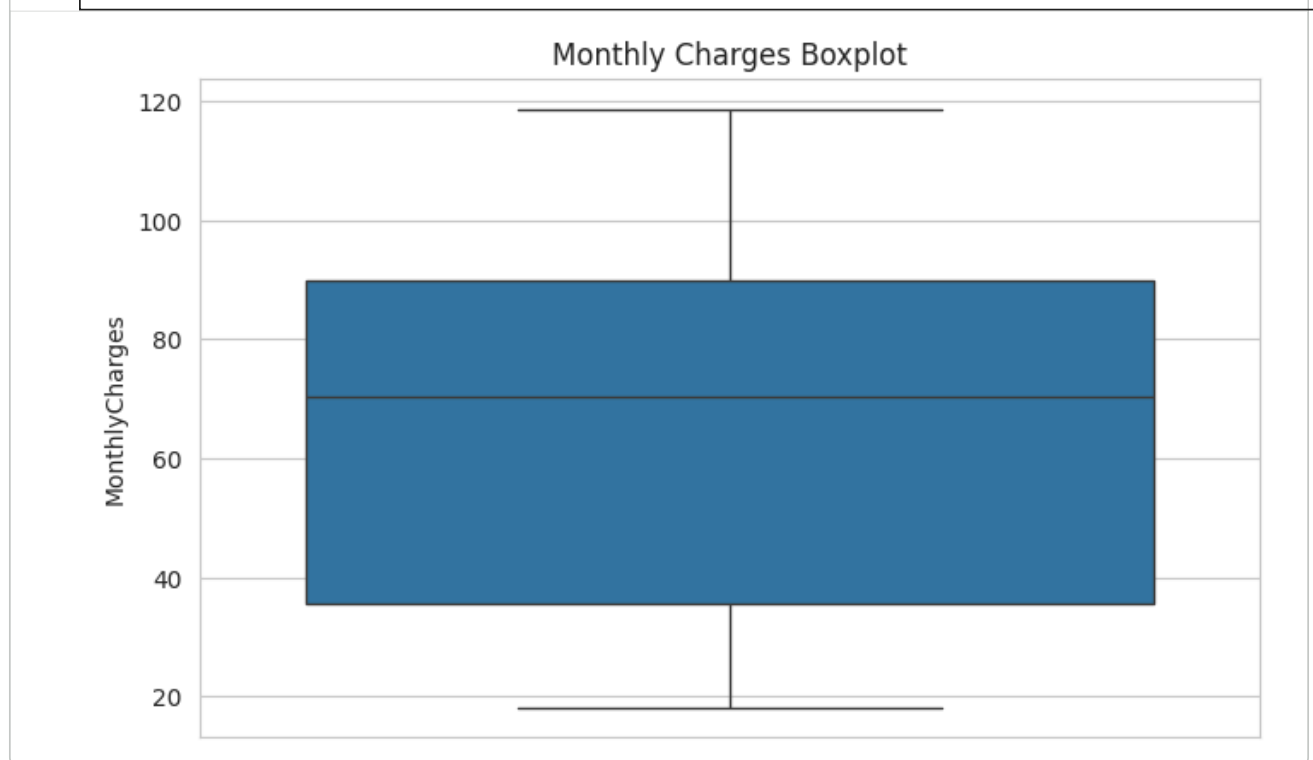
plt.title('Distribution of Customer Tenure')
plt.xlabel('Tenure (Months)')
plt.show()
```



The tenure distribution shows that many customers are either very new or have stayed with the company for a long period. Long-tenure customers generally indicate better retention.

4. Boxplot

```
plt.figure(figsize=(8,5))  
  
sns.boxplot(y=df['MonthlyCharges'])  
  
plt.title('Monthly Charges Boxplot')  
plt.show()
```



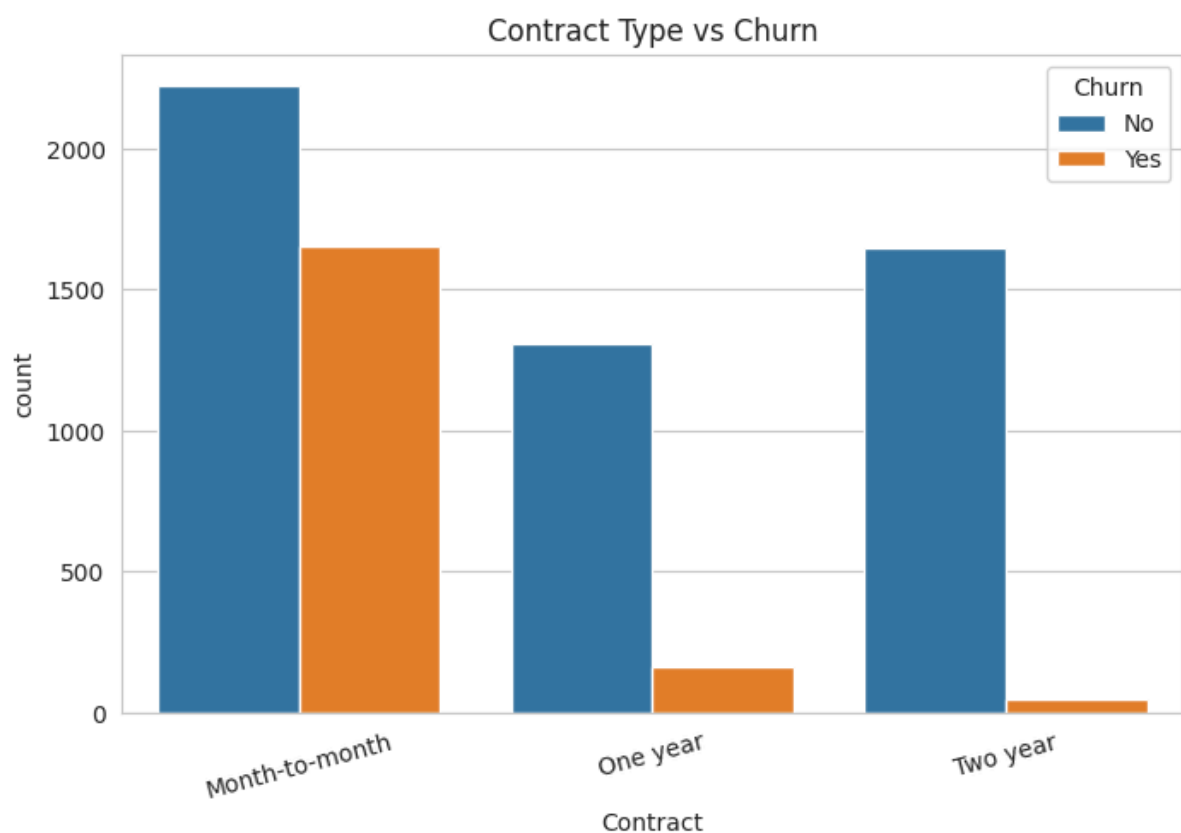
The boxplot helps identify extreme values in MonthlyCharges. A few customers have unusually high monthly charges, indicating potential outliers.

5. Contract Type vs Churn

```
plt.figure(figsize=(8,5))

sns.countplot(
    x='Contract',
    hue='Churn',
    data=df
)

plt.title('Contract Type vs Churn')
plt.xticks(rotation=15)
plt.show()
```



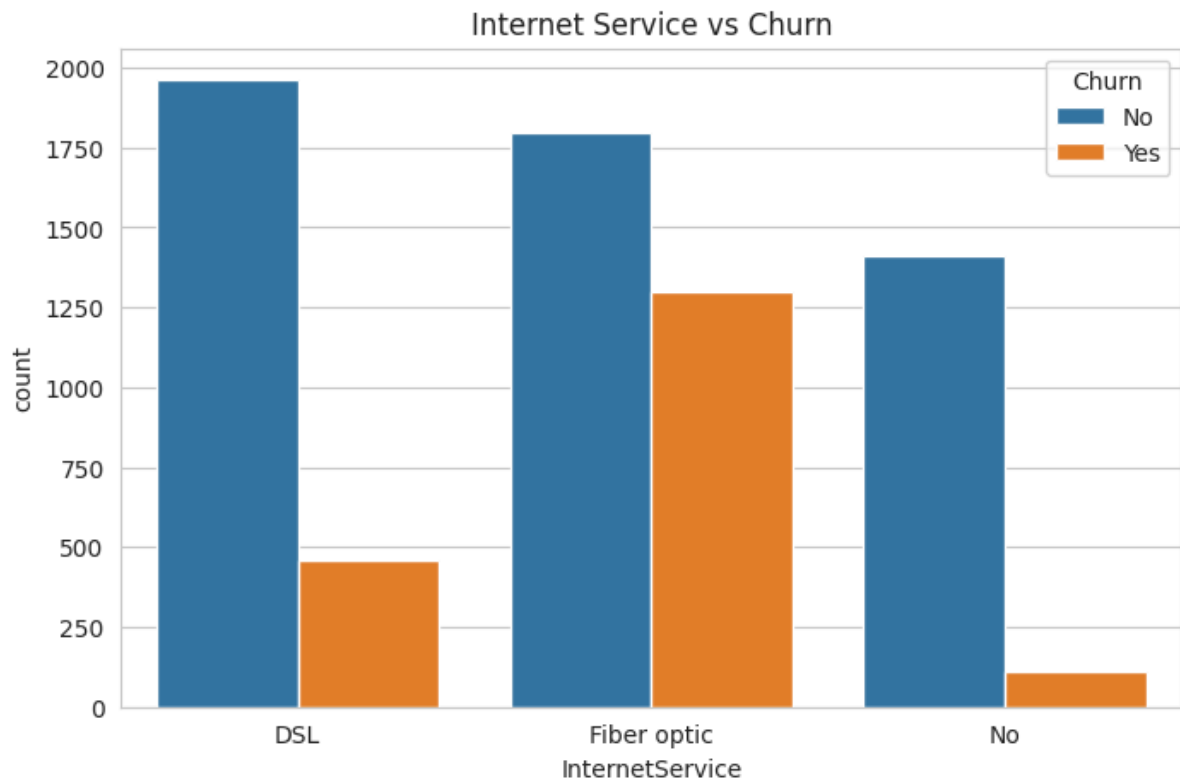
Customers with month-to-month contracts exhibit the highest churn rate. Customers with one-year and two-year contracts are more likely to remain with the company.

6. Internet Service vs Churn


```
plt.figure(figsize=(8,5))

sns.countplot(
    x='InternetService',
    hue='Churn',
    data=df
)

plt.title('Internet Service vs Churn')
plt.show()
```



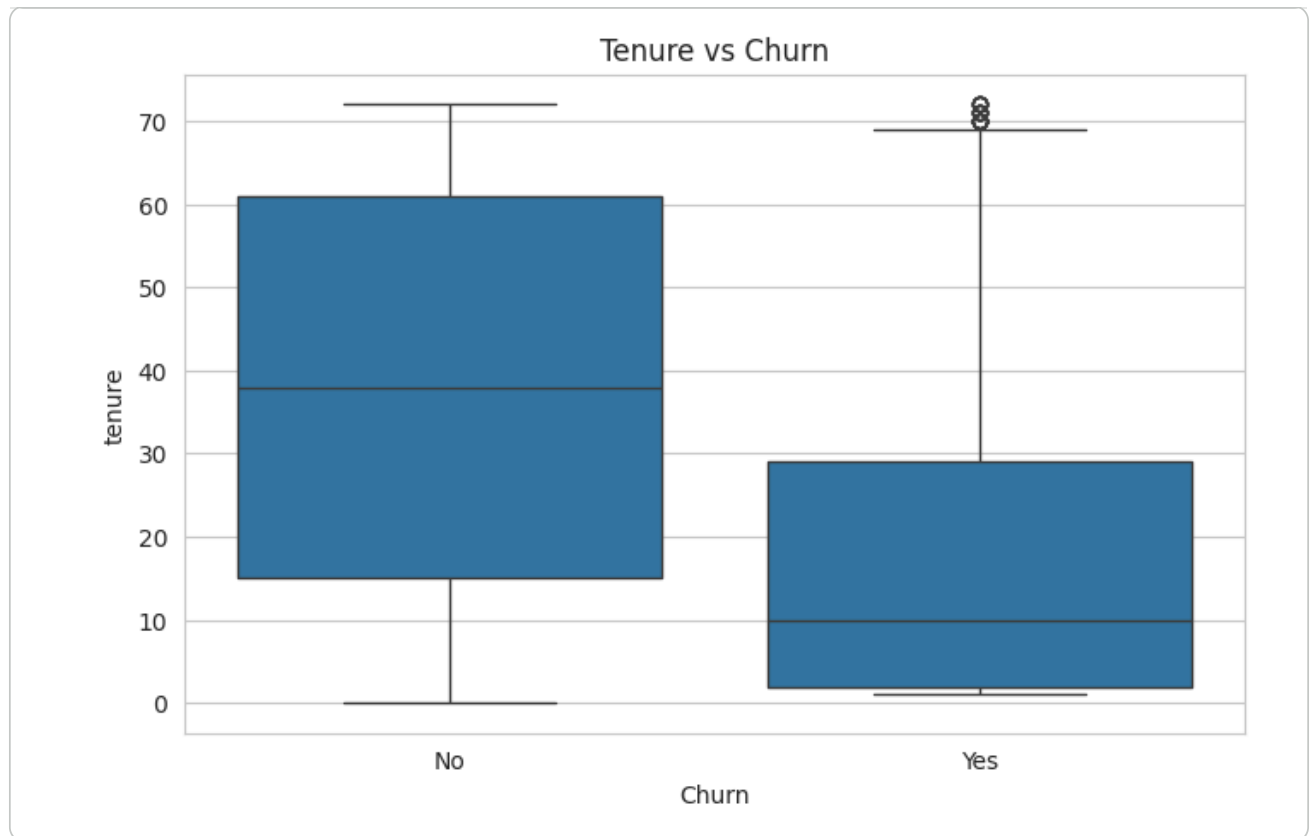
Customers using fiber optic internet services show a higher churn rate compared to DSL users and customers without internet service.

7. Tenure vs Churn

```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='Churn',
    y='tenure',
    data=df
)

plt.title('Tenure vs Churn')
plt.show()
```



Churned customers generally have lower tenure compared to retained customers. This suggests that newer customers are more likely to leave.

8. Correlation Heatmap

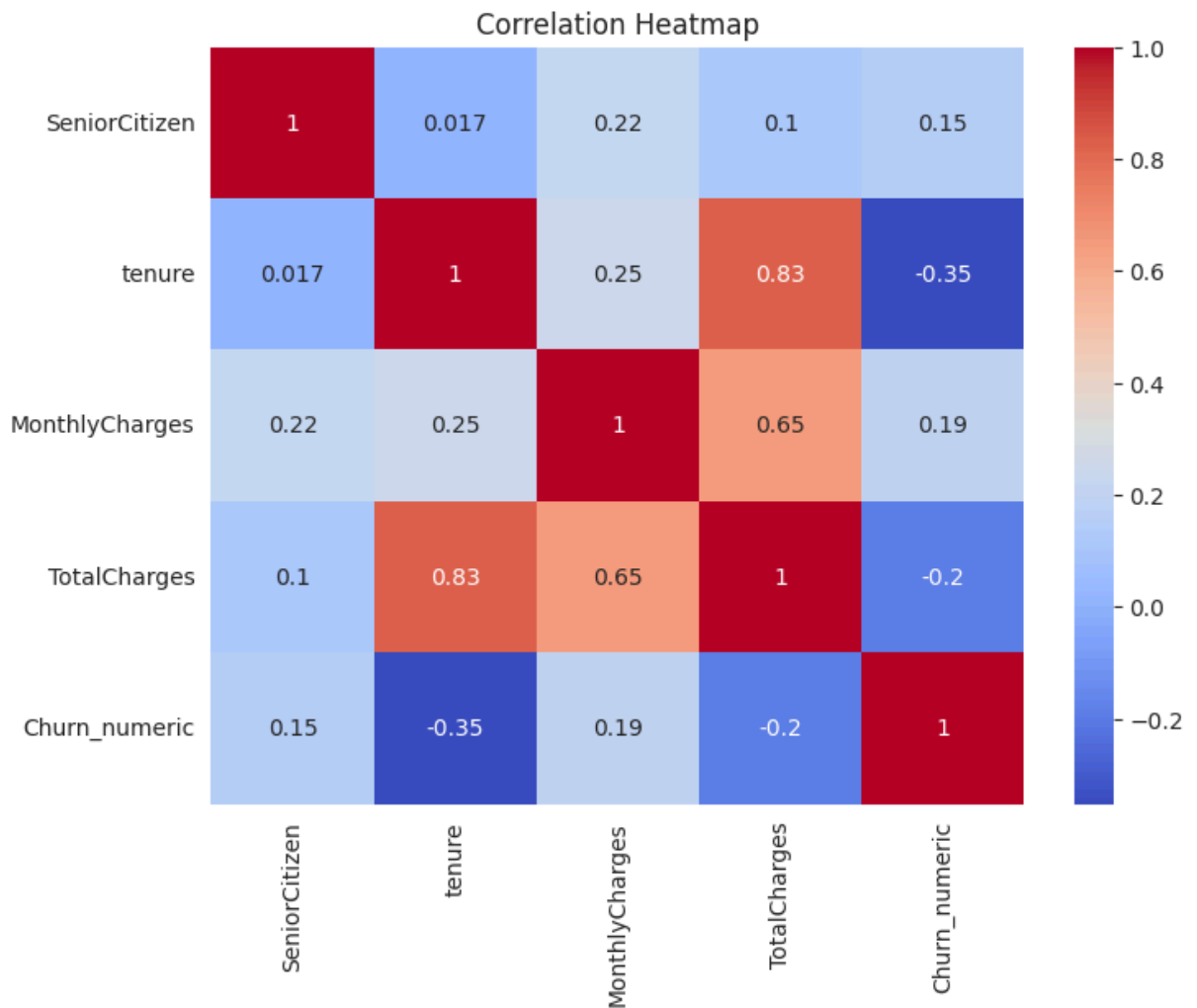
```
df['Churn_numeric'] = df['Churn'].map({
    'Yes':1,
    'No':0
})
```

```
plt.figure(figsize=(8,6))

corr = df[['SeniorCitizen',
           'tenure',
           'MonthlyCharges',
           'TotalCharges',
           'Churn_numeric']].corr()

sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm'
)

plt.title('Correlation Heatmap')
plt.show()
```



Start coding or [generate](#) with AI.

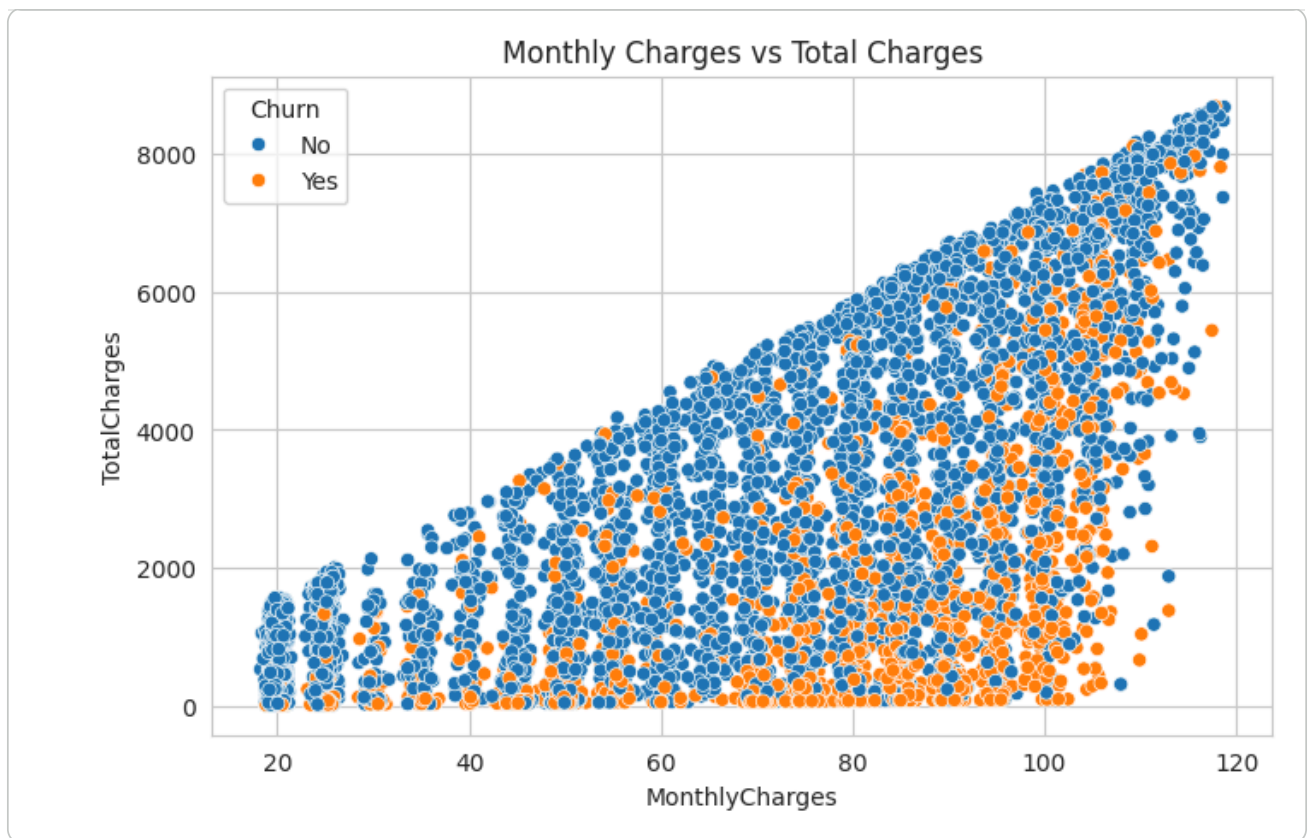
Tenure and TotalCharges show strong positive correlation. Churn has a negative relationship with tenure, indicating that long-term customers are less likely to churn.

9. Scatter Plot

```
plt.figure(figsize=(8,5))

sns.scatterplot(
    x='MonthlyCharges',
    y='TotalCharges',
    hue='Churn',
    data=df
)

plt.title('Monthly Charges vs Total Charges')
plt.show()
```



Customers with high total charges generally have longer tenure. Churned customers are concentrated among customers with lower total charges and shorter service duration.

```
df[['tenure', 'MonthlyCharges', 'TotalCharges']].describe()
```

	tenure	MonthlyCharges	TotalCharges	
count	7043.000000	7043.000000	7043.000000	
mean	32.371149	64.761692	2281.916928	
std	24.559481	30.090047	2265.270398	
min	0.000000	18.250000	18.800000	
25%	9.000000	35.500000	402.225000	
50%	29.000000	70.350000	1397.475000	
75%	55.000000	89.850000	3786.600000	
max	72.000000	118.750000	8684.800000	

The average customer tenure is around 32 months. Monthly charges vary widely among customers, indicating different subscription plans. Total charges exhibit a large range due to differences in tenure and service usage.

```
df.to_csv("Cleaned_Customer_Churn_Dataset_Gundoju_Meher_Sai.csv", index=False)
```

```
from google.colab import files
files.download("Cleaned_Customer_Churn_Dataset_Gundoju_Meher_Sai.csv")
```

1. What percentage of customers churned?

Approximately 26.5% of customers churned.

2. Which customer group has the highest churn?

Customers with Month-to-Month contracts have the highest churn rate.

3. Which numerical features seem related to churn?

Tenure, MonthlyCharges, and TotalCharges.

4. Which categorical features seem related to churn?

Contract Type, Internet Service, Payment Method, and Paperless Billing.

5. Are there missing values or data quality issues?

22 blank values were found in TotalCharges and handled appropriately.

6. Are there outliers in the dataset?

A few outliers were observed in MonthlyCharges.

7. What are the top 3 factors affecting churn?

1. Contract Type
2. Customer Tenure
3. Monthly Charges

8. What recommendations can reduce churn?

- Promote long-term contracts
- Reward loyal customers